

Universidad Autónoma De Yucatán

Facultad de Matemáticas

Ingeniería en Software

Teoria de Lenguajes de Programacion

PI_03. Programacion asincrona, programación logica y funcional

Requisitos de finalización

Presentado Por:

Bolaños Marin Gabriel Emilio

Keb Zi José Carlos

Pech Sanches Jorge Armando

Alejandro Pasos Ruiz

16 de mayo de 2026

Introducción

El presente proyecto consiste en el diseño y la implementación de un servicio web fundamentado en la lógica declarativa. Este sistema funciona como un motor de inferencia simbólica, permitiendo a los usuarios enviar consultas lógicas y obtener resultados derivados de forma automática. La interacción se realiza a través de una API REST desarrollada en el entorno de Node.js utilizando el framework Express, la cual ejecuta consultas sobre una base de conocimiento escrita en el lenguaje Prolog. El objetivo principal de este desarrollo es demostrar la viabilidad y el funcionamiento de la interacción entre distintos paradigmas de programación en una misma arquitectura.

Explicación de paradigmas de programación usados

El sistema logra su objetivo al integrar de manera cohesiva tres paradigmas fundamentales:

- Programación lógica (Prolog): Es el núcleo de la inferencia del sistema. Se utiliza para estructurar la base de conocimiento mediante la definición explícita de hechos y reglas. Esto permite definir las relaciones lógicas y las condiciones necesarias para que el sistema derive nuevas conclusiones.
- Programación funcional (JavaScript): Se aplica en la capa del servidor web. Su uso principal en este contexto es el procesamiento y la normalización de la entrada recibida por el usuario antes de ser evaluada por el motor lógico.
- Programación asíncrona: Aprovecha el modelo de ejecución no bloqueante de Node.js. Esta capa utiliza mecanismos como `async/await` y Promesas (Promises) para gestionar adecuadamente los tiempos de espera del motor y manejar múltiples solicitudes concurrentes sin detener el servidor.

Arquitectura

La arquitectura del sistema está consolidada en un único módulo que integra los siguientes componentes clave:

- Servidor HTTP (Express): Actúa como la interfaz de comunicación con el exterior. Se encarga de exponer un endpoint específico (`/query`) diseñado para recibir las consultas formuladas en formato Prolog.
- Motor de inferencia lógica: Es el componente que evalúa las reglas y los hechos alojados en la base de conocimiento. Está implementado utilizando la biblioteca Tau Prolog.
- Base de conocimiento: Consiste en un archivo escrito en Prolog que actúa como la "memoria" del sistema. Contiene las declaraciones de hechos y las reglas que dictan el comportamiento de inferencia.
- Capa asíncrona: Componente transversal que se asegura de que el servidor mantenga un alto rendimiento al manejar múltiples peticiones simultáneas.

Ejemplo de ejecución

El ciclo de vida de una consulta dentro del sistema sigue un flujo de ejecución estrictamente definido:

1. Petición: El cliente realiza una solicitud HTTP enviando una consulta lógica, como por ejemplo `penalty_applicable(contract1)`.
2. Procesamiento del servidor: El servidor recibe la petición, normaliza la entrada aplicando programación funcional, carga el archivo de la base de conocimiento y procede a inicializar una sesión en Prolog.
3. Evaluación: El motor lógico toma el control, ejecuta la consulta recibida y realiza el proceso de inferencia basándose en las reglas previamente definidas.
4. Respuesta: El resultado de la inferencia lógica es empaquetado y retornado al cliente en formato de respuesta JSON.

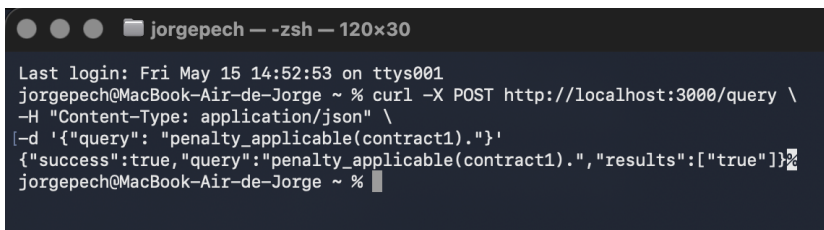
Ejemplo

Prueba 1: Consulta Verdadera

`curl -X POST http://localhost:3000/query \`

`-H "Content-Type: application/json" \`

`-d '{"query": "penalty_applicable(contract1)."}'`



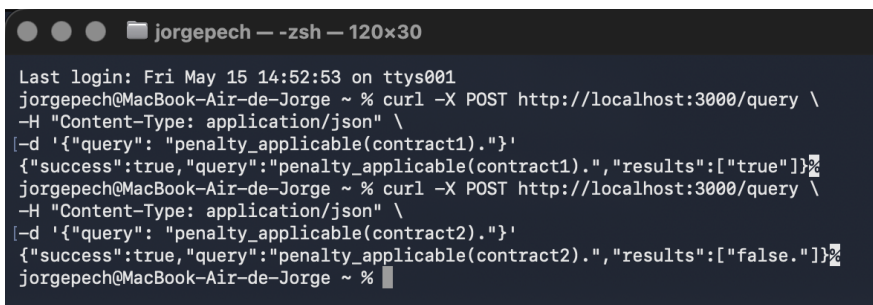
```
jorgepech — zsh — 120x30
Last login: Fri May 15 14:52:53 on ttys001
jorgepech@MacBook-Air-de-Jorge ~ % curl -X POST http://localhost:3000/query \
-H "Content-Type: application/json" \
-d '{"query": "penalty_applicable(contract1)."}'
{"success":true,"query":"penalty_applicable(contract1).","results":["true"]}
jorgepech@MacBook-Air-de-Jorge ~ %
```

Prueba 2: Consulta Falsa (No hay late payment para contract2)

`curl -X POST http://localhost:3000/query \`

`-H "Content-Type: application/json" \`

`-d '{"query": "penalty_applicable(contract2)."}'`



```
jorgepech — zsh — 120x30
Last login: Fri May 15 14:52:53 on ttys001
jorgepech@MacBook-Air-de-Jorge ~ % curl -X POST http://localhost:3000/query \
-H "Content-Type: application/json" \
-d '{"query": "penalty_applicable(contract1)."}'
{"success":true,"query":"penalty_applicable(contract1).","results":["true"]}
jorgepech@MacBook-Air-de-Jorge ~ % curl -X POST http://localhost:3000/query \
-H "Content-Type: application/json" \
-d '{"query": "penalty_applicable(contract2)."}'
{"success":true,"query":"penalty_applicable(contract2).","results":["false."]}
jorgepech@MacBook-Air-de-Jorge ~ %
```

Prueba 3: Buscar todas las variables aplicables (X)

`curl -X POST http://localhost:3000/query \`

`-H "Content-Type: application/json" \`

`-d '{"query": "penalty_applicable(X)."}'`

```
jorgepech — -zsh — 120x30

Last login: Fri May 15 14:52:53 on ttys001
jorgepech@MacBook-Air-de-Jorge ~ % curl -X POST http://localhost:3000/query \
-H "Content-Type: application/json" \
-d '{"query": "penalty_applicable(contract1)."}'
{"success":true,"query":"penalty_applicable(contract1).","results":["true"]}
jorgepech@MacBook-Air-de-Jorge ~ % curl -X POST http://localhost:3000/query \
-H "Content-Type: application/json" \
-d '{"query": "penalty_applicable(contract2)."}'
{"success":true,"query":"penalty_applicable(contract2).","results":["false"]}
jorgepech@MacBook-Air-de-Jorge ~ % curl -X POST http://localhost:3000/query \
-H "Content-Type: application/json" \
-d '{"query": "penalty_applicable(X)."}'
{"success":true,"query":"penalty_applicable(X).","results":["X = contract1"]}
jorgepech@MacBook-Air-de-Jorge ~ %
```

Conclusiones

El desarrollo de este "Motor de Inferencia Lógica como Servicio" valida exitosamente la integración de múltiples paradigmas en una arquitectura moderna. Demuestra que es posible encapsular la complejidad de un motor simbólico (Prolog) detrás de una interfaz web estándar y accesible (Node.js/Express), permitiendo que sistemas externos consuman evaluaciones lógicas complejas de manera sencilla, asíncrona y escalable.

Extensiones a futuro

Persistencia Dinámica: Migrar la base de conocimiento de un archivo de texto estático a una base de datos o permitir la carga dinámica de hechos a través de un endpoint POST.

Seguridad: Implementar autenticación y autorización en el servidor HTTP para restringir quién puede ejecutar consultas lógicas.

Validación de Sintaxis Avanzada: Mejorar la normalización funcional en JavaScript para detectar errores de sintaxis en Prolog antes de enviar la consulta al motor, ahorrando recursos de procesamiento.